

Algorithms for the Hybrid Constrained Longest Common Subsequence Problem

Yi-Ching Chen

Department of Computer Science and Information Engineering,
National Taiwan University, Taipei, Taiwan 106
d94010@csie.ntu.edu.tw

Abstract

We investigate a variant of the longest common subsequence problem. Given two sequences X, Y and two constrained patterns P, Q of lengths m, n, p , and q , respectively, the hybrid constrained longest common subsequence problem is to find a longest common subsequence of X and Y such that the resulting LCS is both a supersequence of P and a nonsupersequence of Q . Without loss of generality, assume that $m \leq n$. We present a new dynamic programming algorithm for solving this problem in $O(mnpq)$ time and space. We also propose another algorithm by restricting the computation on the positions of matches between X and Y . The latter algorithm requires $O(pqr \log \log n + n \log n)$ time over an infinite alphabet and $O((pqr + n) \log \log n)$ time over a finite alphabet, and $O(pq(r + n))$ space for both cases, where r denotes the total number of matches between X and Y .

1 Introduction

A sequence is a string of characters over an alphabet Σ . A subsequence of a sequence X is obtained by deleting zero or more characters from X (not necessarily contiguous). A sequence X is a supersequence of a sequence Y if Y is a subsequence of X , and X is a nonsupersequence of Y if Y is not a subsequence of X . Given a sequence X of length m , let $X[i]$ denote the i th character of X for any $i = 1, \dots, m$. We also let $X[i..j]$ denote the subsequence of consecutive characters in X from position i to position j if $1 \leq i \leq j \leq m$, and an empty string otherwise. For example, if $X = \text{SEQUENCE}$, then $X[7] = \text{C}$ and $X[3..5] = \text{QUE}$.

A longest common subsequence (LCS) of two sequences is a maximum-length common sub-

sequence of the sequences. The LCS problem is to find an LCS of two given sequences. Wagner and Fischer [22] proposed the LCS problem and solved it in quadratic time and space. Hirschberg [13] presented a quadratic-time and linear-space algorithm for the LCS problem. Since then, the LCS problem has been studied intensively to improve the time complexity for decades [1, 2, 4, 10, 12, 14, 15, 18]. However, the LCS problem with an arbitrary number of sequences is NP-hard [17] even on a binary alphabet.

The LCS problem is a special case of the sequence alignment problem [6, 12], and has many applications in computational molecular biology and pattern recognition. One major application is to measure the similarity of biological sequences. In the evolutionary molecular biology, a significant segment of the DNA sequence might have been conserved in different species. To take a common specific segment into account for the similarity measurement, Tsai [20] proposed an inclusion-constrained longest common subsequence (IC-LCS) problem for finding an LCS of two sequences X and Y such that the resulting LCS is a supersequence of P , and gave an $O(m^2n^2p)$ -time algorithm, where m, n , and p are the lengths of X, Y , and P , respectively. Later, Chin et al. [9] and Arslan and Egecioğlu [3] independently presented two improved algorithms with $O(mnp)$ time. Chin et al. [9] also showed that this problem is equivalent to a special case of the constrained multiple sequence alignment problem [8, 19]. Without loss of generality, assume that $m \leq n$. Iliopoulos and Rahman [16] proposed an $O(pr \log \log n + n)$ -time algorithm by employing the van Emde Boas tree [21], where r is the total number of ordered pairs of positions at which X and Y match. It should be noted, however, that the preprocessing time of the van Emde Boas tree was not evaluated within the analysis of the time

complexity.

Recently, Chen and Chao [7] considered a complementary problem of the IC-LCS problem, named exclusion-constrained longest common subsequence (EC-LCS) problem, for finding an LCS of X and Y such that the resulting LCS is a non-supersequence of Q , and solved this problem in $O(mnq)$ time, where q is the length of Q . In this paper, we merge the constraints of the IC-LCS and EC-LCS problems for making the similarity measurement more flexible. The new problem, named the hybrid constrained longest common subsequence (HC-LCS) problem, are formally defined as follows.

Problem (HC-LCS) Given two sequences X, Y and two constrained patterns P, Q of lengths m, n, p , and q , respectively, the HC-LCS problem is to find an LCS of X and Y such that the resulting LCS is both a supersequence of P and a non-supersequence of Q .

Consider sequences $X = \text{BADBABD}$ and $Y = \text{ABCBDDBA}$ as an example. ABBD is an LCS of X and Y . If $P = \text{DA}$ and $Q = \text{BA}$, ADA is a solution of the HC-LCS problem. Both the IC-LCS and EC-LCS problems are obviously special cases of the HC-LCS problem.

In Section 2, we solve the HC-LCS problem by applying the traditional dynamic-programming approach in $O(mnpq)$ time and space. Section 3 presents an improved algorithm for the HC-LCS problem by only considering the positions of matches between X and Y in the dynamic-programming lattices. Let r denote the total number of matches between X and Y . The algorithm runs in $O(pqr \log \log n + n \log n)$ time over an infinite alphabet and $O((pqr + n) \log \log n)$ time over a finite alphabet, and $O(pq(r + n))$ space for both cases. Finally, concluding remarks are given in Section 4.

2 An Algorithm for the HC-LCS Problem

Let $S_{i,j,k,h}$ denote the set of all LCSs of $X[1..i]$ and $Y[1..j]$ where every LCS is both a supersequence of $P[1..k]$ and a non-supersequence of $Q[1..h]$. Lemmas 1 to 5 decompose the structure of an optimal solution based on the solutions to its smaller subproblems.

Lemma 1: If $Z[1..l] \in S_{i,j,k,h}$, where $X[i] =$

$Y[j] = P[k] = Q[h]$, then the following properties hold:

- (a) $h = 1$ implies $Z[1..l]$ is an empty sequence.
- (b) $h \geq 2$ and $Z[l] \neq X[i]$ imply $Z[1..l] \in S_{i-1,j-1,k,h}$.
- (c) $h \geq 2$ and $Z[l] = X[i]$ imply $Z[1..l-1] \in S_{i-1,j-1,k-1,h-1}$.

Proof: First we prove (a). Since $P[k] = Q[h]$ and $h = 1$, $Q[1]$ is a subsequence of $P[1..k]$. If $Z[1..l]$ is not an empty sequence, then $Z[1..l]$ being a supersequence of $P[1..k]$ implies that $Z[1..l]$ is also a supersequence of $Q[1]$. This is a contradiction.

Next we prove (b). Since $Z[l] \neq X[i]$, $Z[1..l]$ is a common subsequence of $X[1..i-1]$ and $Y[1..j-1]$ such that $Z[1..l]$ is a supersequence of $P[1..k]$ and is a non-supersequence of $Q[1..h]$. Assume by contradiction that there exists a common subsequence $C[1..l+1]$ of $X[1..i-1]$ and $Y[1..j-1]$ where $C[1..l+1]$ is a supersequence of $P[1..k]$ and is a non-supersequence of $Q[1..h]$. It follows that $C[1..l+1]$ is also a common subsequence of $X[1..i]$ and $Y[1..j]$ where $C[1..l+1]$ is a supersequence of $P[1..k]$ and is a non-supersequence of $Q[1..h]$, which is a contradiction.

Now we prove (c). Since $Z[l] = X[i] = Y[j] = P[k] = Q[h]$, $Z[1..l-1]$ is a common subsequence of $X[1..i-1]$ and $Y[1..j-1]$ where $Z[1..l-1]$ is a supersequence of $P[1..k-1]$ and is a non-supersequence of $Q[1..h-1]$. Assume by contradiction that there exists a common subsequence $C[1..l]$ of $X[1..i-1]$ and $Y[1..j-1]$ where $C[1..l]$ is a supersequence of $P[1..k-1]$ and is a non-supersequence of $Q[1..h-1]$. If we append $X[i] = Y[j] = P[k] = Q[h]$ to $C[1..l]$, then we obtain a common subsequence of $X[1..i]$ and $Y[1..j]$ of length greater than l , which satisfies the constraints of $P[1..k]$ and $Q[1..h]$. This contradicts the hypothesis. $\square$

We omit the proofs of Lemmas 2 to 5 in the following because they are similar to Lemma 1.

Lemma 2: If $Z[1..l] \in S_{i,j,k,h}$, where $X[i] = Y[j] = P[k]$ and $X[i] \neq Q[h]$, then $Z[l] = X[i]$ and $Z[1..l-1] \in S_{i-1,j-1,k-1,h}$.

Lemma 3: If $Z[1..l] \in S_{i,j,k,h}$, where $X[i] = Y[j] = Q[h]$ and $X[i] \neq P[k]$, then the following properties hold:

- (a) $h = 1$ implies $Z[l] \neq X[i]$ and $Z[1..l] \in S_{i-1,j-1,k,h}$.
- (b) $h \geq 2$ and $Z[l] \neq X[i]$ imply $Z[1..l] \in S_{i-1,j-1,k,h}$.

$$\mathcal{L}(i, j, k, h) = \begin{cases} -\infty & \text{if } X[i] = Y[j] = P[k] = Q[h] \text{ and } h = 1, \\ \max \{ \mathcal{L}(i-1, j-1, k, h), 1 + \mathcal{L}(i-1, j-1, k-1, h-1) \} & \text{if } X[i] = Y[j] = P[k] = Q[h] \text{ and } h \geq 2, \\ 1 + \mathcal{L}(i-1, j-1, k-1, h) & \text{if } X[i] = Y[j] = P[k] \text{ and } (h = 0 \text{ or } X[i] \neq Q[h]), \\ \mathcal{L}(i-1, j-1, k, h) & \text{if } X[i] = Y[j] = Q[h], h = 1, \text{ and } (k = 0 \text{ or } X[i] \neq P[k]), \\ \max \{ \mathcal{L}(i-1, j-1, k, h), 1 + \mathcal{L}(i-1, j-1, k, h-1) \} & \text{if } X[i] = Y[j] = Q[h], h \geq 2, \text{ and } (k = 0 \text{ or } X[i] \neq P[k]), \\ 1 + \mathcal{L}(i-1, j-1, k, h) & \text{if } X[i] = Y[j], (k = 0 \text{ or } X[i] \neq P[k]), \text{ and } (h = 0 \text{ or } X[i] \neq Q[h]), \\ \max \{ \mathcal{L}(i-1, j, k, h), \mathcal{L}(i, j-1, k, h) \} & \text{if } X[i] \neq Y[j]. \end{cases} \quad (1)$$

- (c) $h \geq 2$ and $Z[l] = X[i]$ imply $Z[1..l-1] \in S_{i-1, j-1, k, h-1}$.

Lemma 4: If $Z[1..l] \in S_{i, j, k, h}$, where $X[i] = Y[j]$, $X[i] \neq P[k]$, and $X[i] \neq Q[h]$, then $Z[l] = X[i]$ and $Z[1..l-1] \in S_{i-1, j-1, k, h}$.

Lemma 5: If $Z[1..l] \in S_{i, j, k, h}$, where $X[i] \neq Y[j]$, then the following properties hold:

- (a) $Z[l] \neq X[i]$ implies that $Z[1..l] \in S_{i-1, j, k, h}$.
 (b) $Z[l] \neq Y[j]$ implies that $Z[1..l] \in S_{i, j-1, k, h}$.

Let $\mathcal{L}(i, j, k, h)$ denote the length of an LCS that belongs to $S_{i, j, k, h}$. By the optimal-substructure properties of the HC-LCS problem shown in Lemmas 1 to 5, we have the following recursive formula. The boundary conditions of the recursive formula are $\mathcal{L}(i, 0, 0, h) = \mathcal{L}(0, j, 0, h) = 0$ and $\mathcal{L}(i, 0, k, h) = \mathcal{L}(0, j, k, h) = -\infty$ for any $0 \leq i \leq m$, $0 \leq j \leq n$, $1 \leq k \leq p$, and $0 \leq h \leq q$. For any $0 < i \leq m$, $0 < j \leq n$, $0 \leq k \leq p$, and $0 \leq h \leq q$, $\mathcal{L}$ is computed in order by Equation (1), and the length of an HC-LCS is $\mathcal{L}(m, n, p, q)$. We construct it by backtracking through a path from $\mathcal{L}(m, n, p, q)$ to $\mathcal{L}(0, 0, 0, 0)$. If $X[i] = Y[j]$ and $\mathcal{L}(i, j, k, h)$ equals to $1 + \mathcal{L}(i-1, j-1, k-1, h-1)$, $1 + \mathcal{L}(i-1, j-1, k-1, h)$, $1 + \mathcal{L}(i-1, j-1, k, h-1)$, or $1 + \mathcal{L}(i-1, j-1, k, h)$, we append the character $X[i]$ to the result. Thus, delivering an HC-LCS takes $O(m+n+p+q)$ steps. We have the following theorem.

Theorem 1: The HC-LCS problem can be solved in $O(mnpq)$ time and space.

In fact, the space requirement can be further reduced to $O(pq(m+n))$ by applying Hirschberg's approach [13].

3 An Improved Algorithm for the HC-LCS Problem

During the computation of $\mathcal{L}$ by Equation (1), the $\mathcal{L}$ -values might increase only when a match between X and Y is encountered. To make the computation more efficiently, we restrict it on the positions of matches between X and Y in $\mathcal{L}$.

First of all, the matches between X and Y are calculated. For each position i , we use a linked list $M[i]$ to record all corresponding positions j such that $X[i] = Y[j]$, and keep this list in increasing order.

For all $1 \leq i \leq m$, $1 \leq j \leq n$, $0 \leq k \leq p$, and $0 \leq h \leq q$, let $\mathcal{L}(i, j, k, h) = \mathcal{L}_{k, h}(i, j)$. After the transformation of the $\mathcal{L}$ -notation, $\mathcal{L}$ is visualized as $(p+1) \times (q+1)$ 2-dimensional matrices rather than one 4-dimensional matrix. Equation (1) is reformulated as follows. For any $0 < i \leq m$, $j \in M[i]$, $0 \leq k \leq p$, and $0 \leq h \leq q$,

$$\mathcal{L}_{k, h}(i, j) = \begin{cases} -\infty & \text{if } X[i] = P[k] = Q[h] \text{ and } h = 1, \\ \max \{ \tau_1, 1 + \tau_2 \} & \text{if } X[i] = P[k] = Q[h] \text{ and } h \geq 2, \\ 1 + \tau_3 & \text{if } X[i] = P[k] \text{ and } (h = 0 \text{ or } X[i] \neq Q[h]), \\ \tau_1 & \text{if } X[i] = Q[h], h = 1, \text{ and } (k = 0 \text{ or } X[i] \neq P[k]), \\ \max \{ \tau_1, 1 + \tau_4 \} & \text{if } X[i] = Q[h], h \geq 2, \text{ and } (k = 0 \text{ or } X[i] \neq P[k]), \\ 1 + \tau_1 & \text{if } (k = 0 \text{ or } X[i] \neq P[k]) \text{ and } (h = 0 \text{ or } X[i] \neq Q[h]), \end{cases} \quad (2)$$

where

$$\tau_1 = \max_{1 \leq i' < i, 1 \leq j' < j, j' \in M[i']} \{\mathcal{L}_{k,h}(i', j')\}, \quad (3)$$

$$\tau_2 = \max_{1 \leq i' < i, 1 \leq j' < j, j' \in M[i']} \{\mathcal{L}_{k-1,h-1}(i', j')\}, \quad (4)$$

$$\tau_3 = \max_{1 \leq i' < i, 1 \leq j' < j, j' \in M[i']} \{\mathcal{L}_{k-1,h}(i', j')\}, \quad (5)$$

and

$$\tau_4 = \max_{1 \leq i' < i, 1 \leq j' < j, j' \in M[i']} \{\mathcal{L}_{k,h-1}(i', j')\}. \quad (6)$$

The boundary conditions of this recursive formula are $\mathcal{L}_{0,h}(i, 0) = \mathcal{L}_{0,h}(0, j) = 0$ and $\mathcal{L}_{k,h}(i, 0) = \mathcal{L}_{k,h}(0, j) = -\infty$ for any $0 \leq i \leq m$, $0 \leq j \leq n$, $1 \leq k \leq p$, and $0 \leq h \leq q$.

3.1 The Data Structure

In our algorithm we need to query the element with the highest value (i.e., priority) among all the entries whose j -coordinates (i.e., key) on the computational lattice are below a given threshold. We use a data structure, called a *bounded heap* [5], to support the following operations:

- *Insert*($\mathcal{H}; j, \ell, pos$): Insert into the bounded heap $\mathcal{H}$ the key j with priority ℓ and associated position pos .
- *IncreaseLength*($\mathcal{H}; j, \ell, pos$): If the bounded heap $\mathcal{H}$ does not contain the key j , perform *Insert*($\mathcal{H}; j, \ell, pos$); otherwise, set the priority of this key to $\max\{\ell, \ell'\}$, where ℓ' is its original priority.
- *BoundedMax*($\mathcal{H}; j$): Return the maximum priority of all items in $\mathcal{H}$ with key smaller than j and its associated position. If $\mathcal{H}$ does not contain any items with keys smaller than j , return null.

The bounded heap uses the van Emde Boas tree [21] as a kernel structure. It follows that the initialization of the bounded heap takes $O(n \log \log n)$ time, and each of the above operations requires $O(\log \log n)$ time, where keys are drawn from the set $\{1, \dots, n\}$ [5].

3.2 The Algorithm

We first sort X and Y , and then scan the two sorted sequences to build a linked list $M[i]$, for each position i of X , as an increasing list of all corresponding positions j such that $X[i] = Y[j]$. Note that the original 4-dimensional matrix $\mathcal{L}$ is

visualized as $(p+1) \times (q+1)$ 2-dimensional matrices, i.e., $\mathcal{L}_{k,h}$ where $0 \leq k \leq p$ and $0 \leq h \leq q$. We maintain a bounded heap $\mathcal{H}_{k,h}$ for each $\mathcal{L}_{k,h}$ to implement Equations (3) to (6).

Each $\mathcal{L}_{k,h}$ is computed by proceeding matches between X and Y in row-major order. While proceeding row i of $\mathcal{L}_{k,h}$, $\mathcal{H}_{k,h}$ has kept a position with its $\mathcal{L}_{k,h}$ -value for each $1 \leq j' \leq n$ such that the position provides the maximum $\mathcal{L}_{k,h}$ -value among all positions (i', j', k, h) where $1 \leq i' < i$, and empty if no such position exists. For each j in $M[i]$, we query the position that has the maximum $\mathcal{L}_{k,h}$ -value among all positions in $\mathcal{H}_{k,h}$ with key smaller than j , and then store its length in $T_{k,h}(i, j)$. According to Equation (2), we compute $\mathcal{L}_{k,h}(i, j)$ by the relationships among $X[i]$, $P[k]$, and $Q[h]$, and then place the position (i, j, k, h) with $\mathcal{L}_{k,h}(i, j)$ into the insert list $\mathcal{I}$. After proceeding row i of $\mathcal{L}_{k,h}$, $\mathcal{H}_{k,h}$ is updated by performing *IncreaseLength* on each element of $\mathcal{I}$. Finally, we query the position that has the maximum $\mathcal{L}_{k,h}$ -value among all positions in $\mathcal{H}_{p,q}$ with key smaller than $n+1$. The maximum value in $\mathcal{H}_{p,q}$ is the length of an HC-LCS, and an HC-LCS can be obtained by tracing back through the backtracking links from the position with the maximum length until $i = 0$ or $j = 0$. The algorithm is drawn in Figure 1.

Since sorting is invoked in building the m linked lists in line 1 of Algorithm HC-LCS, it takes $O(n \log n)$ time over an infinite alphabet and $O(n)$ time over a finite alphabet, and $O(n)$ space [10]. The initialization of each $\mathcal{H}_{k,h}$ in line 2 takes $O(n \log \log n)$ time. Let r denote the total number of matches between X and Y . In lines 4–23, we need to maintain an $\mathcal{H}_{k,h}$ and $\mathcal{I}$ for each $\mathcal{L}_{k,h}$, and there are at most r insertions and queries on $\mathcal{H}_{k,h}$. Thus, computing $\mathcal{L}_{k,h}$ takes $O(r \log \log n)$ time and $O(r+n)$ space. Consequently, the calculation of the $(p+1) \times (q+1)$ matrices requires in total $O(pqr \log \log n)$ time and $O(pq(r+n))$ space. In line 24, querying on $\mathcal{H}_{p,q}$ takes $O(\log \log n)$ time, and delivering an HC-LCS takes $O(m+n+p+q)$ steps. In summary, we have the following theorem.

Theorem 2: Given two sequences X, Y and two constrained patterns P, Q of lengths m, n, p , and q , respectively. Without loss of generality, assume that $m \leq n$. Let r denote the total number of matches between X and Y .

1. Algorithm HC-LCS runs in $O(pqr \log \log n + n \log n)$ time and $O(pq(r+n))$ space over an infinite alphabet.

Algorithm HC-LCS(X, Y, P, Q)

```

1: Build a linked list  $M[i]$  for all  $1 \leq i \leq m$ ;
2: Initialize the bounded heap  $\mathcal{H}_{k,h}$  for all  $1 \leq k \leq p$  and  $1 \leq h \leq q$ ;
3: Perform  $Insert(\mathcal{H}_{k,h}; 0, 0, (0, 0, k, h))$  if  $k = 0$  and  $Insert(\mathcal{H}_{k,h}; 0, -\infty, (0, 0, k, h))$  otherwise;
4: for  $k \leftarrow 0$  to  $p$  do
5:   for  $h \leftarrow 0$  to  $q$  do // compute each  $\mathcal{L}_{k,h}$ 
6:     for  $i \leftarrow 1$  to  $m$  do
7:       Initialize the insert list  $\mathcal{I}$  as an empty list;
8:       for each  $j$  in  $M[i]$  do
9:          $T_{k,h}(i, j) \leftarrow BoundedMax(\mathcal{H}_{k,h}; j)$ ; // the 2nd, 4th, and 5th cases of Equation (2)
10:         $\mathcal{L}_{k,h}(i, j) \leftarrow T_{k,h}(i, j)$ ;
11:        if  $X[i] = P[k]$  and  $X[i] = Q[h]$  and  $h = 1$  then
12:           $\mathcal{L}_{k,h}(i, j) \leftarrow -\infty$ ; // the 1st case of Equation (2)
13:        else if  $X[i] = P[k]$  and  $X[i] = Q[h]$  and  $h \geq 2$  and  $T_{k,h}(i, j) \leq 1 + T_{k-1,h-1}(i, j)$  then
14:           $\mathcal{L}_{k,h}(i, j) \leftarrow 1 + T_{k-1,h-1}(i, j)$ ; // the 2nd case of Equation (2)
15:        else if  $X[i] = P[k]$  and  $(X[i] \neq Q[h] \text{ or } h = 0)$  then
16:           $\mathcal{L}_{k,h}(i, j) \leftarrow 1 + T_{k-1,h}(i, j)$ ; // the 3rd case of Equation (2)
17:        else if  $X[i] = Q[h]$  and  $h \geq 2$  and  $(X[i] \neq P[k] \text{ or } k = 0)$  and  $T_{k,h}(i, j) \leq 1 + T_{k,h-1}(i, j)$  then
18:           $\mathcal{L}_{k,h}(i, j) \leftarrow 1 + T_{k,h-1}(i, j)$ ; // the 5th case of Equation (2)
19:        else if  $(X[i] \neq P[k] \text{ or } k = 0)$  and  $(X[i] \neq Q[h] \text{ or } h = 0)$  then
20:           $\mathcal{L}_{k,h}(i, j) \leftarrow 1 + T_{k,h}(i, j)$ ; // the 6th case of Equation (2)
21:         $\mathcal{I} \leftarrow \mathcal{I} \cup (\mathcal{L}_{k,h}(i, j), (i, j, k, h))$ ;
22:        for each  $(\mathcal{L}_{k,h}(i, j), (i, j, k, h))$  on  $\mathcal{I}$  do
23:           $IncreaseLength(\mathcal{H}_{k,h}; j, \mathcal{L}_{k,h}(i, j), (i, j, k, h))$ ;
24: Output  $BoundedMax(\mathcal{H}_{p,q}; n + 1)$ ;
```

Figure 1: An algorithm for the HC-LCS problem.

2. Algorithm HC-LCS runs in $O((pqr + n) \log \log n)$ time and $O(pq(r + n))$ space over a finite alphabet.

4 Concluding Remarks

In this paper, we studied the HC-LCS problem, which is a hybrid problem of the IC-LCS and EC-LCS problems. We presented an algorithm for solving the HC-LCS problem in $O(mnpq)$ time and space. We also proposed another algorithm, named Algorithm HC-LCS, by restricting the computation on the positions of matches between X and Y , which requires $O(pqr \log \log n + n \log n)$ time over an infinite alphabet and $O((pqr + n) \log \log n)$ time over a finite alphabet, and $O(pq(r + n))$ space for both cases. In the worse case, Algorithm HC-LCS requires $O(pqmn \log \log n + n \log n)$ time over an infinite alphabet or $O(pqmn \log \log n)$ time over a finite alphabet. However, Algorithm HC-LCS outperforms the traditional dynamic programming algorithm if r is less than $n(\frac{m}{\log m} - 1)$.

For the case where the number of constrained patterns is arbitrary, Gotthilf et al. [11] showed

that the IC-LCS problem is NP-hard and does not have a polynomial-time approximation scheme (PTAS). In fact, the HC-LCS problem with multiple constrained patterns can be further shown being NP-hard.

Acknowledgments

This work was supported in part by NSC grants NSC 95-2221-E-002-126-MY3 and NSC 97-2221-E-002-097-MY3 from the National Science Council, Taiwan. The author would like to express sincere thanks to her advisor Prof. Kun-Mao Chao, as well as Justie Su-Tzu Juan and Guan-Shieng Huang, for the valuable comments and suggestions.

References

- [1] A.V. Aho, D.S. Hirschberg, and J.D. Ullman. Bounds on the complexity of the longest common subsequence problem. *Journal of ACM*, 23:1–12, 1976.

- [2] A. Apostolico and C. Guerra. The longest common subsequence problem revisited. *Algorithmica*, 2:315–336, 1987.
- [3] A.N. Arslan and Ö. Eğecioğlu. Algorithms for the constrained longest common subsequence problems. *International Journal of Foundations of Computer Science*, 16(6):1099–1109, 2005.
- [4] L. Bergroth, H. Hakonen, and T. Raita. A survey of longest common subsequence algorithms. In *Proceedings of the 7th International Symposium on String Processing and Information Retrieval (SPIRE'00)*, pp. 39–48, 2000.
- [5] G.S. Brodal, K. Kaligosi, I. Katriel, and M. Kutz. Faster algorithms for computing longest common increasing subsequence. In *Proceedings of the 17th Annual Symposium on Combinatorial Pattern Matching (CPM'2006)*, pp. 330–341, 2006.
- [6] K.M. Chao and L. Zhang. *Sequence Comparison: Theory and Methods*, Springer, 2009.
- [7] Y.C. Chen and K.M. Chao. *On the generalized constrained longest common subsequence problems*, *Journal of Combinatorial Optimization*, accepted.
- [8] F.Y.L. Chin, N.L. Ho, T.W. Lam, and P.W.H. Wong. Efficient constrained multiple sequence alignment with performance guarantee. *Journal of Bioinformatics and Computational Biology*, 3(1):1–18, 2005.
- [9] F.Y.L. Chin, A.D. Santis, A.L. Ferrara, N.L. Ho, and S.K. Kim. A simple algorithm for the constrained longest common sequence problems. *Information Processing Letters*, 90:175–179, 2004.
- [10] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein. *Introduction to Algorithms*, second edition, The MIT Press and McGraw-Hill, MA, 2001.
- [11] Z. Gotthilf, D. Hermelin, and M. Lewenstein. Constrained LCS: hardness and approximation. In *Proceedings of the 19th Annual Symposium on Combinatorial Pattern Matching (CPM'08)*, pp. 255–262, 2008.
- [12] D. Gusfield. *Algorithms on Strings, Trees, and Sequences*, Cambridge University Press, Cambridge, MA, 1997.
- [13] D.S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of ACM*, 18:341–343, 1975.
- [14] D.S. Hirschberg. Algorithms for the longest common subsequence problem. *Journal of ACM*, 24 (1977) 664–675.
- [15] J.W. Hunt and T.G. Szymanski. A fast algorithm for computing longest common subsequence. *Communications of ACM*, 20(5):350–353, 1977.
- [16] C.S. Iliopoulos and M.S. Rahman. New efficient algorithms for the LCS and constrained LCS problems. *Information Processing Letters*, 106:13–18, 2008.
- [17] D. Maier. The complexity of some problems on subsequences and supersequence. *Journal of ACM*, 25:322–336, 1978.
- [18] W.J. Masek and M.S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20:18–31, 1980.
- [19] C.Y. Tang, C.L. Lu, M.D. Chang, Y.T. Tsai, Y.J. Sun, K.M. Chao, J.M. Chang, Y.H. Chiou, C.M. Wu, H.T. Chang, and W.I. Chou. Constrained multiple sequence alignment tool development and its application to RNase family alignment. *Journal of Bioinformatics and Computational Biology*, 1(2):267–287, 2003.
- [20] Y.T. Tsai. The constrained longest common subsequence problem. *Information Processing Letters*, 88:173–176, 2003.
- [21] P. van Emde Boas. Preserving order in a forest in less than logarithmic time and linear space. *Information Processing Letters*, 6:80–82, 1977.
- [22] R.A. Wagner and M.J. Fischer. The string-to-string correction problem. *Journal of ACM*, 21(1):168–173, 1974.